SIMATS ENGINEERING — ASSESSMENT TOOL 2

COURSE CODE:	MLA03	**COURSE NAME:**	Reinforcement Learning
COURSE OUTCOME:	CO4: Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. (BL3, BL4, BL5)		
FACULTY:	Dr. G. A. SENTHIL	**TOTAL MARKS:**	50 Marks (40% Weightage) Duration: 4 Hrs

MODULE 1: MODEL-BASED RL FOUNDATIONS

Q1. Flowchart of Model-Based RL Workflow (Environment Interaction to Policy Optimization) [5 Marks]

Workflow Explanation: Model-Based Reinforcement Learning systematically alternates between real-world exploration, parameter estimation of the transition dynamics & reward functions, trajectory rollouts (simulated imagination), and policy/value improvement.

```
+----------------------+      Action a_t      +----------------------+
|  Real Environment    |<------------------  |    Agent Policy      |
|  Dynamics P(s'|s,a)  | ------------------> |    pi_theta(a|s)     |
+----------------------+  Next State s_{t+1},  +----------------------+
         |           Reward r_t                  ^
         v                                       | Policy
   [Replay Buffer D]                             | Optimization
         |                                       |
         v Transitions (s, a, r, s')             |
+--------------------------------+               |
|  Model Learning Component      |               |
|  Train f_phi(s,a) ≈ s', r_psi(s,a)|            |
+--------------------------------+               |
         | Learned Model Dynamics                |
         v                                       |
+--------------------------------+               |
|  Planning / Rollout Generation  | -------------------+
|  Simulate Trajectories & Optimize |
+--------------------------------+
```

• **Step 1: Real-World Rollout:** Agent interacts with environment using current policy π_θ and stores transition tuples $(s\_t, a\_t, r\_t, s\_\{t+1\})$ into replay buffer D.
• **Step 2: Supervised Model Fitting:** Learn transition dynamics model f_φ by minimizing regression loss: $L(\varphi) = E\_\{(s,a,s')\sim D\} [|| f\_\varphi(s,a) - s' ||^2]$.
• **Step 3: Simulation & Optimization:** Generate virtual trajectories via f_φ and update policy parameters θ using either sampling-based trajectory planning or analytic policy gradients.

Q2. Concept Map: Model-Based RL vs. Model-Free RL Comparison [5 Marks]

Evaluation Dimension	Model-Free RL (e.g., PPO, SAC, DQN)	Model-Based RL (e.g., MBPO, PETS, MuZero)
Environment Knowledge	Direct trial-and-error; learns value/policy directly without explicit state transition dynamics.	Explicitly models state transitions s_{t+1} ~ P_hat(s'|s,a) and rewards R_hat(s,a).
Sample Efficiency	Low (requires millions of real environment interactions to converge).	High (orders of magnitude fewer real samples; leverages virtual rollouts/imagination).
Computational Complexity	Lower compute per interaction step; fast policy updates.	High compute overhead due to iterative model fitting, trajectory forecasting, and planning.
Asymptotic Bias & Limits	Unbiased convergence to optimal policy; superior asymptotic performance.	Prone to model exploitation and compounding prediction error bias over long horizons.

MODULE 2: ANALYTIC GRADIENT COMPUTATION

Q3. Flowchart: Analytic Gradient Computation & Backpropagation Through Dynamics [5 Marks]

Core Mechanism: When both the transition dynamics model s_{t+1} = f_φ(s_t, a_t) and policy a_t = π_θ(s_t) are parameterized by differentiable neural networks, the gradient of the total expected return ∇_θ J(θ) can be computed analytically by unrolling the computation graph over horizon H and applying Backpropagation Through Time (BPTT).

```
Initial State s_0
  |
  +--> [Policy pi_theta] --> Action a_0 --> [Model f_phi] --> State s_1 --> Reward r_0(s_0, a_0)
                                      |
                                      +--> [Policy pi_theta] --> a_1 --> [Model f_phi] --> s_2 ...
=================================================================================
                  BACKWARD PASS (Chain Rule / BPTT)
<-- dJ/dtheta = sum_{t=0}^H [ (dr_t/ds_t)(ds_t/dtheta) + (dr_t/da_t)(da_t/dtheta) ] <-- Accumulate Gradients <--
```

- **Forward Trajectory Pass:** Starting from s_0, roll out deterministic path: a_t = π_θ(s_t), s_{t+1} = f_φ(s_t, a_t), r_t = r(s_t, a_t) for t = 0 ... H-1.
- **Recursive Chain Rule:** The state Jacobian propagates backward recursively: ds_{t+1}/dθ = (∂f/∂s_t)(ds_t/dθ) + (∂f/∂a_t)[(∂π/∂s_t)(ds_t/dθ) + ∂π/∂θ].
- **Parameter Update:** θ ← θ + α ∇_θ J(θ) using first-order (Adam) or quasi-Newton gradient ascent.

Q4. Block Diagram: Policy Parameter Updates, Gradients, and Reward Optimization [5 Marks]

```
+--------------------------------------------------------------------------------+
|                        TRAJECTORY OPTIMIZATION LOOP                     |
|                                                              |
| +-------------------+      s_t, a_t    +-------------------+                   |
| | Differentiable     | --------------> | Loss Formulation  |                  |
| | Forward Model f_phi|                 | L(theta) = -J(theta)                 |
| +-------------------+                +-------------------+                  |
```

```
|          | State Jacobians (df/ds, df/da)      |                          |
|          v                              v                       |
| +----------------------------------------------------------+                |
| |  Analytic Gradient Engine (BPTT / Pathwise Derivative)     |                |
| |  grad_theta J = grad_theta sum_{t=0}^H gamma^t r(s_t, pi(s_t))            |
| +----------------------------------------------------------+                |
|                         | Gradient Vector g                          |
|                         v                                   |
| +----------------------------------------------------------+                |
| |  Optimizer (Adam / L-BFGS / Natural Policy Gradient)       |                |
| |  theta_{k+1} = theta_k + alpha * Update(g, H^{-1})        |                |
| +----------------------------------------------------------+                |
+------------------------------------------------------------------------------+
```

Mitigation of Exploding/Vanishing Gradients: Analytic gradients over long horizons suffer from chaotic and unstable dynamics. Modern approaches resolve this using trajectory collocation, short truncated rollouts, or regularized value baselines.

MODULE 3: SAMPLING-BASED PLANNING

Q5. Flowchart: Sampling-Based Planning Process (Model Predictive Control & CEM) [5 Marks]

Overview: Sampling-based planners evaluate action sequences in simulation without requiring differentiable transitions, making them highly robust to discontinuities and non-convex environments.

```
[Current State s_t]
      |
      v
+----------------------------------------------------------+
| 1. Action Sequence Sampling: Sample K candidate sequences    |
|    A^(k) = (a_t^(k), a_{t+1}^(k), ..., a_{t+H-1}^(k)) ~ N(mu, Sigma) |
+----------------------------------------------------------+
      |
      v
+----------------------------------------------------------+
| 2. Forward Trajectory Rollout using Learned Dynamics f_hat   |
|    s_{tau+1}^(k) = f_hat(s_tau^(k), a_tau^(k)) for tau = t...t+H-1 |
+----------------------------------------------------------+
      |
      v
+----------------------------------------------------------+
| 3. Trajectory Evaluation: Compute Total Cumulative Return    |
|    G^(k) = sum_{tau=t}^{t+H-1} gamma^{tau-t} r_hat(s_tau^(k), a_tau^(k)) |
+----------------------------------------------------------+
      |
      v
+----------------------------------------------------------+
| 4. Selection / Distribution Refinement (CEM):               |
|    Select Top-Elite trajectories; update distribution (mu, Sigma) |
+----------------------------------------------------------+
      |
      v
+----------------------------------------------------------+
| 5. Receding Horizon Execution: Execute a_t^*; Discard Rest;   |
|    Observe next real state s_{t+1} and Re-plan              |
+----------------------------------------------------------+
```

Q6. Concept Map: Exhaustive Search Planning vs. Sampling-Based Planning [5 Marks]

Dimension	Exhaustive Search (e.g., Minimax, BFS/DFS)	Sampling-Based Planning (e.g., MPC, CEM, MCTS)
Search Space Coverage	Evaluates all $|A|^H$ possible branches across the complete decision tree.	Samples a stochastic subset $K \ll |A|^H$ of high-probability candidate paths.
Action Space Compatibility	Strictly limited to small discrete action spaces; intractable for continuous control.	Natively handles continuous, high-dimensional control spaces via Gaussian distributions.
Curse of Dimensionality	Suffers exponential computational explosion: $O(|A|^H)$.	Scales polynomial/linearly with sample size: $O(K * H * T_{eval})$.
Optimality Guarantees	Guarantees exact globally optimal path across evaluated horizon H.	Asymptotically optimal or bounded sub-optimal depending on sample density K.

MODULE 4: MODEL-BASED DATA GENERATION

Q7. Block Diagram & Flowchart: Dyna-Architecture & Synthetic Data Generation [5 Marks]

Dyna Framework (Sutton): Unifies direct reinforcement learning (learning directly from real environment transitions) and indirect model-based learning (generating synthetic experience replay via the learned dynamics model).

```
      +------------------------+
      |    Real Environment    |
      +------------------------+
         ^              | Real Experience
Action a |              v (s, a, r, s')
      +------------------------+    +----------------------+
      |        Agent           | <---- | Real Replay Buffer D_env|
      | (Q-Learning / Actor-Critic)|    +----------------------+
      +------------------------+              |
         ^            ^                 v
         | Q-updates        | Q-updates     +-----------------+
         | from simulated rollouts| from real data| Dynamics Model   |
         |                |             | T_hat(s,a)->s', r |
      +------------------------+         +-----------------+
      | Synthetic Buffer D_model | <-----------------+  (Generates branch
      +------------------------+                rollouts of length k)
```

• **Model Training:** Learns transition model T_hat on real dataset D_env.
• **Branched Rollouts:** Starts from real sampled states $s \sim D_{env}$, rolls out k-step simulated steps using policy π, and caches them in D_model.
• **Policy Update:** Samples mini-batches with mixing ratio $\beta\, D_{env} + (1-\beta)\, D_{model}$ to update Q-functions or Actor networks.

Q8. Concept Map: Advantages and Limitations of Model-Generated Training Data [5 Marks]

Category	Key Mechanisms	Impact on RL Performance
Advantages	• Sample Efficiency: Multiplies data volume 10x-100x. • Safe Exploration: Tests risky actions in simulation. • High Parallelism: GPU-batched vectorized rollouts.	Dramatically reduces costly and hazardous real-world hardware interactions.
Limitations & Pitfalls	• Compounding Model Errors: Errors accumulate exponentially over horizon H. • Model Exploitation: Policy exploits inaccurate high-reward regions. • Distribution Shift: Policy visits out-of-distribution states.	Can lead to severe policy degradation unless bounded by short rollouts (k-step) and probabilistic ensembles.

MODULE 5: VALUE-EQUIVALENCE & POLICY OPTIMIZATION (MBPO)

Q9. Block Diagram: Model-Based Policy Optimization (MBPO) Framework [5 Marks]

MBPO Framework (Janner et al.): Guarantees monotonic policy improvement by combining bootstrapped probabilistic neural network ensembles (to quantify epistemic uncertainty) with short k-step branched rollouts initialized from real environment states.

```
+------------------------------------------------------------------------------+
|                    MBPO ALGORITHM WORKFLOW PIPELINE                    |
|                                                         |
| 1. REAL ENVIRONMENT INTERACTION:                                        |
|    Sample s_{t+1} ~ P(s'|s_t, a_t) using policy pi_theta. Store in D_env.     |
|                                                         |
| 2. PROBABILISTIC ENSEMBLE MODEL TRAINING:                                   |
|    Train ensemble {f_phi_1, ..., f_phi_B} on D_env with Gaussian NLL loss:     |
|    L(phi_i) = sum [ log sigma_phi(s,a) + (s' - mu_phi(s,a))^2 / (2*sigma_phi^2) ] |
|                                                         |
| 3. SHORT-HORIZON BRANCHED ROLLOUTS:                                      |
|    Sample initial state s ~ D_env. Perform k-step simulated rollout with pi_theta.|
|    Add synthetic transitions (s, a, r, s') to D_model.                  |
|                                                         |
| 4. SAC POLICY & VALUE UPDATE:                                          |
|    Sample batch from D_env U D_model; update Critic Q_psi & Actor pi_theta via SAC.|
+------------------------------------------------------------------------------+
```

Q10. Comprehensive Concept Map: Unified Model-Based RL Framework [5 Marks]

```
                      +-----------------------------------------+
                      |     UNIFIED MODEL-BASED RL FRAMEWORK     |
                      +-----------------------------------------+
                                      |
      +------------------------+--------------+------------------------+
      |                    |                            |
      v                    v                            v
```

```
+-----------------+      +------------------+                  +------------------+
| 1. MODEL        |      | 2. PLANNING &    |                  | 3. POLICY        |
|   LEARNING      |      |   SEARCH         |                  |   OPTIMIZATION   |
+-----------------+      +------------------+                  +------------------+
| • Maximum       |      | • Sampling-Based |                  | • Analytic       |
|   Likelihood    |      |   (MPC, CEM, MCTS)|                 |   Gradients (BPTT)|
| • Probabilistic |      | • Value-         |                  | • Dyna / Synthetic|
|   Ensembles     |      |   Equivalence    |                  |   Replay (MBPO)  |
| • Latent Models |      |   (MuZero/Dreamer)|                 | • Actor-Critic   |
|   (World Models)|      | • Trajectory Tree |                 |   Updates (SAC)  |
+-----------------+      +------------------+                  +------------------+
       \                    |                             /
        \                   |                            /
         +----------------------->  OPTIMAL AGENT  POLICY <----------------+
                            pi^*(a|s) = argmax E[R]
```

Synthesis: This unified architecture illustrates how model learning, value-equivalent latent dynamics, and policy optimization integrate to overcome sample inefficiency and compounding error in complex decision-making tasks.

Code of Conduct:

I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Student Name: K. BALAMURALI   Register Number: 192472010

Signature of the Student: K. Balamurali   Date: 25-08-2026

RUBRICS FOR CALCULATION

Criteria	Weight (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	**30%**	Highly accurate, in-depth, and insightful technical explanation.	Technically correct content with adequate depth and clarity.	Basic concepts presented with limited accuracy.	Major technical errors; poor understanding.
Problem Identification & Formulations	**25%**	Rigorously formulated algorithms with mathematical clarity and design flow.	Clearly defined approach with logical engineering flow.	Methodology or algorithmic steps are weak.	Unclear or incorrect statements.
Use of Tools & Visuals	**20%**	Advanced block diagrams, flowcharts, and structured concept maps.	Clear and effective diagrams used.	Limited or basic diagrammatic representation.	Minimal or uninformative visuals.
Communication & Organization	**15%**	Structured, professional, concise, and logically sequenced.	Organized and clear technical communication.	Understandable but lacks clean structure.	Poor organization; disjointed sections.
Professionalism & Integrity	**10%**	Complete adherence to academic integrity and rubric standards.	Professional presentation and adherence.	Minor formatting or integrity gaps.	Lacks professional formatting.